

# Code Explanation: Sentiment Analysis Preprocessing Pipeline

## Preprocessing Dictionaries

```
1 CONTRACTIONS = {
2     "ain't": "am not", "aren't": "are not", ...
3 }
4
5 SLANG_CORRECTIONS = {
6     "naw": "no", "nah": "no", "aint": "am not", ...
7 }
8
9 EMOJI_MAP = {
10     '😊': 'happy ', '😄': 'happy ', ...
11 }
```

Defines three lookup dictionaries to standardize textual input:

- **CONTRACTIONS**: Maps English contractions to their full forms.
- **SLANG\_CORRECTIONS**: Converts common internet slang and abbreviations into standard English.
- **EMOJI\_MAP**: Translates unicode emojis into descriptive text tokens to preserve emotional context.

## Preprocessing Helper Functions

```
1 def expand_contractions(text):
2     for contraction, replacement in CONTRACTIONS.items():
3         text = re.sub(r'\b' + re.escape(contraction) + r'\b', replacement, text, flags=re.
4             IGNORECASE)
5     return text
6
7 def expand_slang(text):
8     for slang, replacement in SLANG_CORRECTIONS.items():
9         text = re.sub(r'\b' + re.escape(slang) + r'\b', replacement, text, flags=re.
10             IGNORECASE)
11     return text
12
13 def replace_emojis(text):
14     for emoji, desc in EMOJI_MAP.items():
15         text = text.replace(emoji, desc)
16     # ... regex to remove remaining emojis ...
17     return text
```

Implements functions to iterate through the previously defined dictionaries. These functions utilize regular expressions to identify patterns (contractions, slang, emojis) within the input text and replace them with their standardized counterparts.

## Negation Handling

```
1 def handle_negations(text):
2     text = re.sub(r'\b(not|no|never|none|nobody|nothing|neither|nor)\s+(\w+)', r'\1_2',
3         text)
4     return text
```

Detects negation words (e.g., "not", "never") and merges them with the subsequent word using an underscore. This preserves context (e.g., transforming "not good" into "not\_good"), preventing the loss of negative sentiment during tokenization.

## Main Preprocessing Pipeline

```
1 def preprocess_text(text):
2     if not isinstance(text, str) or len(text.strip()) == 0:
3         return ""
4
5     text = expand_contractions(text)
6     text = expand_slang(text)
7     text = replace_emojis(text)
8     text = text.lower()
9     text = re.sub(r'<.*?>', ' ', text) # HTML tags
10    text = re.sub(r'http\S+|www\S+', ' ', text) # URLs
11    text = re.sub(r'@\w+', ' ', text) # Mentions
12    text = handle_negations(text)
13    text = re.sub(r'\d+', ' <NUM> ', text) # Numbers
14    text = re.sub(r'[^a-z\s!?\'\_]', ' ', text) # Special chars
15    text = re.sub(r'\s+', ' ', text).strip()
16
17    return text
```

Orchestrates the cleaning process for a single text string. The pipeline includes type checking, lowercase conversion, removal of HTML/URLs/handles, application of dictionaries, number tokenization, and stripping of non-alphabetic characters.

## Dataset Processing

```
1 def preprocess_full_dataset(filepath='sentiment_dataset.csv'):
2     df = pd.read_csv(filepath)
3     df = df[df['text'].notna() & (df['text'].astype(str).str.strip() != '')].copy()
4
5     df['text_clean'] = df['text'].apply(preprocess_text)
6     df = df[df['text_clean'].str.strip() != ''].copy()
7
8     df['text_length'] = df['text_clean'].apply(lambda x: len(x.split()))
9     max_length = int(df['text_length'].quantile(0.95))
10
11    return df, max_length
```

Loads the raw CSV dataset and performs batch cleaning. It handles missing values, applies the `preprocess_text` function to the entire corpus, and calculates the 95th percentile of sentence length to determine an optimal truncation threshold.

## Vocabulary Class

```
1 class Vocabulary:
2     def __init__(self, max_size=1500000, min_freq=2):
3         self.word2idx = {'<PAD>': 0, '<UNK>': 1, '<NUM>': 2}
4         self.idx2word = {0: '<PAD>', 1: '<UNK>', 2: '<NUM>'}
5         self.word_freq = Counter()
6
7     def build_vocab(self, texts):
8         # ... counts frequencies and populates mappings ...
9
10    def encode(self, text, max_length):
11        tokens = text.split()
12        indices = [self.word2idx.get(token, self.word2idx['<UNK>']) for token in tokens[:
max_length]]
```

```

13         if len(indices) < max_length:
14             indices += [self.word2idx['<PAD>']] * (max_length - len(indices))
15         return indices

```

Manages the mapping between text tokens and integer indices.

- **build\_vocab**: Constructs the dictionary based on word frequency, respecting a minimum frequency threshold.
- **encode**: Converts a text string into a fixed-length list of integers, handling truncation and padding.

## Dataset Class

```

1 class SentimentDataset(Dataset):
2     def __init__(self, texts, labels, vocab, max_length):
3         # ... stores data ...
4
5     def __getitem__(self, idx):
6         text = self.texts[idx]
7         label = self.labels[idx]
8         indices = self.vocab.encode(text, self.max_length)
9
10        return {
11            'input_ids': torch.tensor(indices, dtype=torch.long),
12            'attention_mask': torch.tensor([1 if i != 0 else 0 for i in indices], dtype=
13            torch.long),
14            'label': torch.tensor(label, dtype=torch.long)
15        }

```

Implements a custom PyTorch Dataset. It retrieves data samples by index, encodes the text using the **Vocabulary**, and returns tensors for input IDs, attention masks (identifying non-padding tokens), and labels.

## Data Loader Creation

```

1 def create_data_loaders(df, max_length=10000000, batch_size=64, test_size=0.2, random_state
2     =42):
3     train_df, test_df = train_test_split(..., stratify=df['label'])
4
5     vocab = Vocabulary(...)
6     vocab.build_vocab(train_df['text_clean'].tolist())
7
8     train_dataset = SentimentDataset(...)
9     test_dataset = SentimentDataset(...)
10
11    train_loader = DataLoader(..., shuffle=True)
12    test_loader = DataLoader(..., shuffle=False)
13
14    return train_loader, test_loader, vocab, train_df, test_df

```

Handles the operational setup for training. It performs a stratified train-test split, builds the vocabulary exclusively from training data to avoid data leakage, and initializes PyTorch DataLoaders for batched iteration.

## Discriminant Analysis Calculation

```

1 # ... (Frequency counting code) ...
2
3 for word in all_words:
4     # ... count retrieval ...
5     if total_count >= min_freq:
6         ratio_neg = (neg_count + 1) / (total_count - neg_count + 1)

```

```

7     ratio_neu = (neu_count + 1) / (total_count - neu_count + 1)
8     ratio_pos = (pos_count + 1) / (total_count - pos_count + 1)
9     # ... storage ...

```

Analyzes the corpus to identify words that are highly specific to a single sentiment class. It calculates a "discriminant ratio" for each word, comparing its frequency in one class against its frequency in all other classes combined.

## Visualization

```

1 fig, axes = plt.subplots(3, 2, figsize=(16, 18))
2
3 # ... plotting code for Negative, Neutral, Positive ...
4
5 plt.tight_layout()
6 plt.show()

```

Generates a grid of bar charts. The visualization compares the "discriminant words" (most unique to a sentiment) against the "frequent words" (most common overall) for each of the three sentiment classes.

## Tools Used

- **re**: Used for regular expression operations in text cleaning.
- **pandas**: Used for data manipulation and CSV file handling.
- **numpy**: Used for numerical operations.
- **torch**: The core PyTorch library for tensor computations.
- **torch.utils.data**: Used for creating the Dataset and DataLoader abstractions.
- **collections**: Used for counting word frequencies.
- **sklearn.model\_selection**: Used for splitting data into training and testing sets.
- **random**: Used for random number generation operations.
- **matplotlib.pyplot**: Used for generating the visualization plots.